Personal Information

Name: Maja Murawka

StudentID: **13755129**

Email: **maja.murawka@student.uva.nl**

Github: majmurm/predicting-science-at-risk

Submitted on: **23.03.2026**

Data Context

> Research Question: **To what extent can the risk of adverse outcomes for publicly funded research be predicted using textual and metadata features?**

This EDA convers data used to predict the vulnerability of publically funded research in the United States to adverse political outcomes - specifically, grant termination and dataset deletion. The context is the ongoing disruption to U.S. federal data infrastructure under the Trump administration, where executive orders have led to removal of datasets, restriction of access to federal web pages, and terminations of research grants at agencies including the National Institutes of Health (NIH), National Science Foundation (NSF), Environmental Protection Agency (EPA), Centres for Disease Control and Prevention (CDC), and Substance Abuse and Mental Health Services Administration (SAMHSA).

Data Description

Two primary sources are used.

The first is the Grant Witness website, which tracks defunded federal research grants and provides structured metadata such as titles, abstracts, award amounts, activity dates, and termination status. Grant Witness aggregates their data from multiple independent sources, including submissions from affected scientists, news reports, official agency websites, and public budget estimations.

The second is the Data Rescue Project (DRP) Portal, a volunteer-led archive of federal datasets considered at risk of deletion. DRP metadata was scraped

from the project's Github repository using `requests` library. Their actions are based on tips from researchers or other stakeholders, focused on agencies targeted in Project 2025 or in a presidential executive order.

In [123…
```
# Imports
import os
import warnings
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

from urllib.parse import urlparse
from sklearn.manifold import TSNE
sns.set_theme(style='whitegrid')

warnings.filterwarnings('ignore', category=FutureWarning, module='s
warnings.filterwarnings('ignore', category=PendingDeprecationWarnin
```

In [124…
```
from IPython.display import display, HTML
display(HTML("""
<style>
    table { font-size: 9px !important; border-collapse: collapse; }
    th, td { padding: 2px 6px !important; white-space: nowrap; }
    @media print {
        table { font-size: 7px !important; }
        th, td { padding: 1px 4px !important; }
    }
</style>
"""))
```

Data Loading

Grant Witness

The Grant Witness dataset was created by stacking datasets from five federal agencies (CDC, EPA, NIH, NSF, SAMHSA) on shared columns:

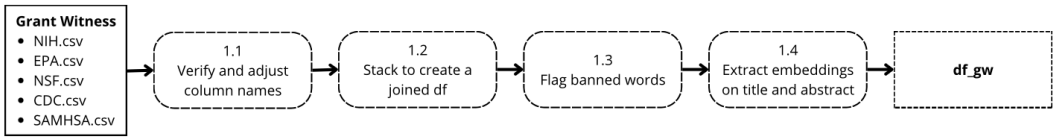
- status,
- title,
- abstract,
- award amounts,
- termination details.

Each row represents a grant within an agency.

I manually adjusted small naming inconsistencies (for example, `organization` to `org_name`, or `award_value` to `award_amount`). The

resulting dataframe consists of 12,156 rows.

The figure below represents data preprocessing performed on Grant Witness dataset:



In [125…
```
#GW
df_gw = pd.read_csv('grant_witness/00_data/df_full.csv')
print(df_gw.shape)
df_gw.head(5)
```

```
(12156, 41)
```

Out[125…

	Unnamed: 0	abstract	agency	award_amount	award_outlaid	award_remaining	embeddings_abstract_allminilm	embeddin
0	0	NaN	NIH	69867044.0	26075910.60	43791133.40	[-1.18838333e-01 4.82986905e-02 -2.54816213e-...	
1	1	NaN	NIH	15533132.0	9874704.00	5658428.00	[-1.18838333e-01 4.82986905e-02 -2.54816213e-...	
2	2	NaN	NIH	22010380.0	3884147.00	18126233.00	[-1.18838333e-01 4.82986905e-02 -2.54816213e-...	
3	3	PROJECT SUMMARY This institutional grant propo...	NIH	8000000.0	8000000.00	0.00	[-7.46721849e-02 -1.63383186e-02 -8.39300230e-...	
4	4	Project Summary/Abstract Wards 7 and 8 in Wash...	NIH	2000000.0	618137.31	1381862.69	[-1.29834609e-02 2.11151000e-02 -1.03895999e-...	

5 rows × 41 columns

Data Rescue Project

The DRP data was scraped from the DRP Github repository using `requests` library. The original dataset consists of 2,972 entries, and each row represents a dataset rescued by DRP. It includes metadata columns: file title, organisation, agency, websites, data source, description, last modification date, category, url, format, status, donwload date, notes, and maintainer.

In [126…
```
df_drp_scraped = pd.read_csv('data_rescue_project/00_data/df_scrape
df_drp_scraped.head(5)
```

Out [126…

	Unnamed: 0	Unnamed: 0_x	file	schema	title	organization	agency	websites	
0	0	0	_datasets/10j-injunctions.md	data_rescue_project	10(j) Injunctions	National Labor Relations Board	National Labor Relations Board	nlrb.gov	ht
1	1	1	_datasets/1998-2023-serotype-data-for-invasive...	data_rescue_project	1998-2023 Serotype Data for Invasive Pneumococ...	Centers for Disease Control and Prevention (CDC)	Department of Health and Human Services	data.cdc.gov	https:/
2	2	2	_datasets/2002-2024-mbda-grantees.md	data_rescue_project	2002-2024 MBDA Grantees	Minority Business Development Agency	Department of Commerce	mbda.gov	https://www.ml
3	3	3	_datasets/2006-iur-public-database.md	data_rescue_project	2006 IUR Public Database	Environmental Protection Agency	Environmental Protection Agency	epa.gov	https://
4	4	4	_datasets/2013-2014-phap-associates-by-state.md	data_rescue_project	2013-2014 PHAP Associates by State	Centers for Disease Control and Prevention (CDC)	Department of Health and Human Services	data.cdc.gov	https://data

5 rows × 25 columns

Most of the metadata fields are related to republishing and maintenance, with little information on the datasets themselves (aside from title). Fields that could contain this information, like description or notes, or additional metadata, are mostly NaN values.

In [127…
```
df_drp_scraped.isna().mean().sort_values(ascending=False).head(10)
```

Out [127…
```
description      0.996692
metadata_url     0.953688
notes            0.718492
format           0.175984
size             0.105855
download_date    0.043004
url              0.014886
agency           0.000331
websites         0.000331
data_source      0.000331
dtype: float64
```

In [128…
```
df_drp_scraped[['description', 'notes', 'category', 'metadata_availa
```

Out [128…

	description	notes	category	metadata_available
0	NaN	Section 10(j) of the National Labor Relations ...	Labor & Employment	True
1	NaN	NaN	Health & Healthcare	False
2	NaN	NaN	Business & Economy	False
3	NaN	There was a 2006 version and a 2002-1986 versi...	Climate & Environment	True
4	NaN	NaN	Health & Healthcare	False

Therefore, I decided to scrape any additional metadata from DataCite REST API. As only 49 datasets have an actual DOI as url, it was necessary to extract DOI from other urls (domains incl. DataLumos, Harvard Dataverse, Urban Data Catalog, Zenodo) using `regex`.

In [129…
```
df_drp_scraped["domain"] = df_drp_scraped["url"].apply(lambda u: ur
df_drp_scraped['domain'].value_counts().head(5)
```

Out [129…

```
domain
www.datalumos.org        2586
www.dropbox.com            85
sciop.net                  72
doi.org                    49
dataverse.harvard.edu      46
Name: count, dtype: int64
```

This allowed to acquire additional metadata for a total of 2539 of 2970 DRP datasets (loaded as `df_drp`).

In [130…

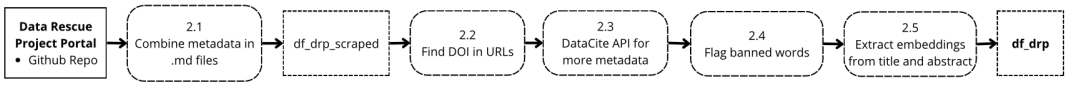
```
df_drp = pd.read_csv('data_rescue_project/00_data/drp_withdoi.csv')
df_drp.drop(columns=['Unnamed: 0'], inplace=True)
df_drp.head(5)
```

Out [130…

	file	schema	title	organization	agency	websites	data_sou
0	_datasets/10j-injunctions.md	data_rescue_project	10(j) Injunctions	National Labor Relations Board	National Labor Relations Board	nlrb.gov	https://www.nlrb.gov/what-v do/investigate-c
1	_datasets/1998-2023-serotype-data-for-invasive...	data_rescue_project	1998-2023 Serotype Data for Invasive Pneumococ...	Centers for Disease Control and Prevention (CDC)	Department of Health and Human Services	data.cdc.gov	https://data.cdc.gov/Public-Heal Surveillan
2	_datasets/2002-2024-mbda-grantees.md	data_rescue_project	2002-2024 MBDA Grantees	Minority Business Development Agency	Department of Commerce	mbda.gov	https://www.mbda.gov/research/data/mb grant
3	_datasets/2006-iur-public-database.md	data_rescue_project	2006 IUR Public Database	Environmental Protection Agency	Environmental Protection Agency	epa.gov	https://www.epa.gov/chemical-da reporting/d
4	_datasets/2013-2014-phap-associates-by-state.md	data_rescue_project	2013-2014 PHAP Associates by State	Centers for Disease Control and Prevention (CDC)	Department of Health and Human Services	data.cdc.gov	https://data.cdc.gov/dataset/2013-20 PHAP-A

5 rows × 106 columns

The figure below provides an summary of preprocessing steps for DRP data:



Feature engineering

As mentioned on the pipeline figures, both datasets were enriched with additional features based on presence of flagged words. Based on two banned words lists published by PEN and by NYT, the script analyses words in abstracts and titles, resulting in the following:

- binary flag for detected banned words: `has_flagged_word` / `has_flagged_word_title` / `has_flagged_word_total`,
- total number of banned words: `n_banned_terms` / `n_banned_terms_title` / `n_banned_terms_total`,
- lists of unique banned words: `flagged_words_pen` / `flagged_words_pen_title`, `flagged_words_nyt` / `flagged_words_nyt_title`, `flagged_words_all` / `flagged_words_all_title`,

- number of unqiue banned words: `num_flagged_words` / `num_flagged_words_title` / `num_flagged_words_total`, `num_pen_words` / `num_pen_words_title` / `num_pen_words_total`, `num_nyt_words` / `num_nyt_words_title` / `num_nyt_words_total`.

Embeddings for titles and abstracts were generated using all-MiniLM-L6-v2, a sentence transformed model. They are stored in columns `embeddings_abstract_allminilm` and `embeddings_title_allminilm`.

Grant Witness

Analysis 1: Missing data

In [131…

```
missing_by_agency = df_gw.groupby('agency').apply(lambda g: g.isna(
missing_by_agency = missing_by_agency.loc[:, (missing_by_agency > 0
missing_by_agency.T.style.format('{:.1%}').background_gradient(cmap
```

```
/var/folders/f0/g61h031s7yscf43_m6xj3l0c0000gn/T/ipykernel_98469/130
8206892.py:1: FutureWarning: DataFrameGroupBy.apply operated on the
grouping columns. This behavior is deprecated, and in a future versi
on of pandas the grouping columns will be excluded from the operatio
n. Either pass `include_groups=False` to exclude the groupings or ex
plicitly select the grouping columns after groupby to silence this w
arning.
  missing_by_agency = df_gw.groupby('agency').apply(lambda g: g.isn
a().mean())
```

Out[131…

agency	CDC	EPA	NIH	NSF	SAMHSA
abstract	100.0%	0.0%	0.4%	0.3%	8.9%
award_amount	0.6%	0.0%	0.3%	0.2%	0.1%
award_outlaid	2.2%	11.3%	1.2%	0.1%	4.4%
award_remaining	2.2%	11.3%	1.2%	0.1%	4.4%
embeddings_abstract_allminilm	100.0%	0.0%	0.0%	0.0%	0.0%
flagged_words_all	100.0%	6.1%	5.5%	8.0%	11.2%
flagged_words_all_title	59.0%	76.2%	78.6%	50.2%	54.7%
flagged_words_nyt	100.0%	23.5%	30.0%	13.4%	15.4%
flagged_words_nyt_title	85.4%	93.0%	83.4%	57.9%	58.5%
flagged_words_pen	100.0%	6.1%	19.4%	8.0%	11.2%
flagged_words_pen_title	59.0%	76.2%	78.8%	50.3%	54.7%
org_city	0.0%	0.0%	0.3%	0.3%	0.0%
org_name	0.0%	0.0%	0.3%	0.2%	0.0%
org_state	0.4%	0.0%	0.5%	0.2%	0.0%
project_title	0.0%	0.0%	0.3%	0.2%	0.0%
reinstatement_date	56.1%	88.4%	56.8%	68.3%	4.6%
reinstatement_indicator	56.1%	88.5%	56.8%	68.3%	4.6%
start_date_original	0.6%	0.0%	50.8%	0.2%	0.1%
termination_date	9.8%	0.4%	43.0%	0.0%	0.0%
usaspending_url	0.6%	0.0%	0.0%	0.0%	0.1%

Many columns have missing values across all agencies. For example `reinstatement_date` and `reinstatement_indicator` are only populated for grants that were reinstated or unfrozen, and `abstract` is entirely absent for CDC, which is why it has to be excluded for analysis of textual (abstract-based) features.

For other agencies the share of missing values in `abstract` column is insignificant.

In [132…
```
df_gw.groupby('agency')['abstract'].apply(lambda x: x.isna().mean()
```

Out[132…
```
agency
CDC       1.000000
EPA       0.000000
NIH       0.004322
NSF       0.002505
SAMHSA    0.089389
Name: abstract, dtype: float64
```

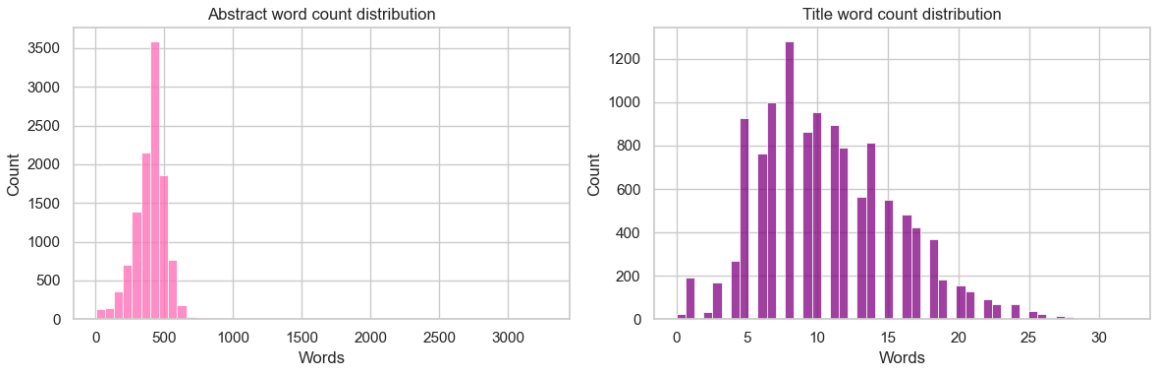
In [133…
```
df_gw['abstract_word_count'] = df_gw['abstract'].fillna('').astype(
df_gw['title_word_count'] = df_gw['project_title'].fillna('').astyp
df_gw_nona = df_gw.loc[df_gw['abstract'].notna()].copy()

desc = pd.concat([df_gw['title_word_count'].describe(), df_gw['abst
desc.columns = ['title_wc_all', 'abstract_wc_all', 'abstract_wc_no_
desc
```

Out[133…

	title_wc_all	abstract_wc_all	abstract_wc_no_na
count	12156.000000	12156.000000	11321.000000
mean	10.728858	369.085884	396.308453
std	4.875921	150.138454	115.822304
min	0.000000	0.000000	2.000000
25%	7.000000	313.000000	335.000000
50%	10.000000	405.000000	414.000000
75%	14.000000	458.000000	462.000000
max	32.000000	3285.000000	3285.000000

Titles word counts in GW have a mean of 10.7, and min of 0 and maximum of 32. For abstracts (on the full GW df, including NaN values) the mean word count is 369 with standard deviation of 150, and with exclusion of NaN values - 396 with standard deviation of 116.

In [134…
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(data=df_gw_nona, x='abstract_word_count', bins=50, ax=
axes[0].set_title('Abstract word count distribution')
axes[0].set_xlabel('Words')
sns.histplot(data=df_gw, x='title_word_count', bins=50, ax=axes[1],
axes[1].set_title('Title word count distribution')
axes[1].set_xlabel('Words')
plt.tight_layout()
plt.show()
```



In [135…
```
print((df_gw_nona['abstract_word_count'] > 1000).value_counts())
print((df_gw_nona['title_word_count'] > 25).value_counts())
```

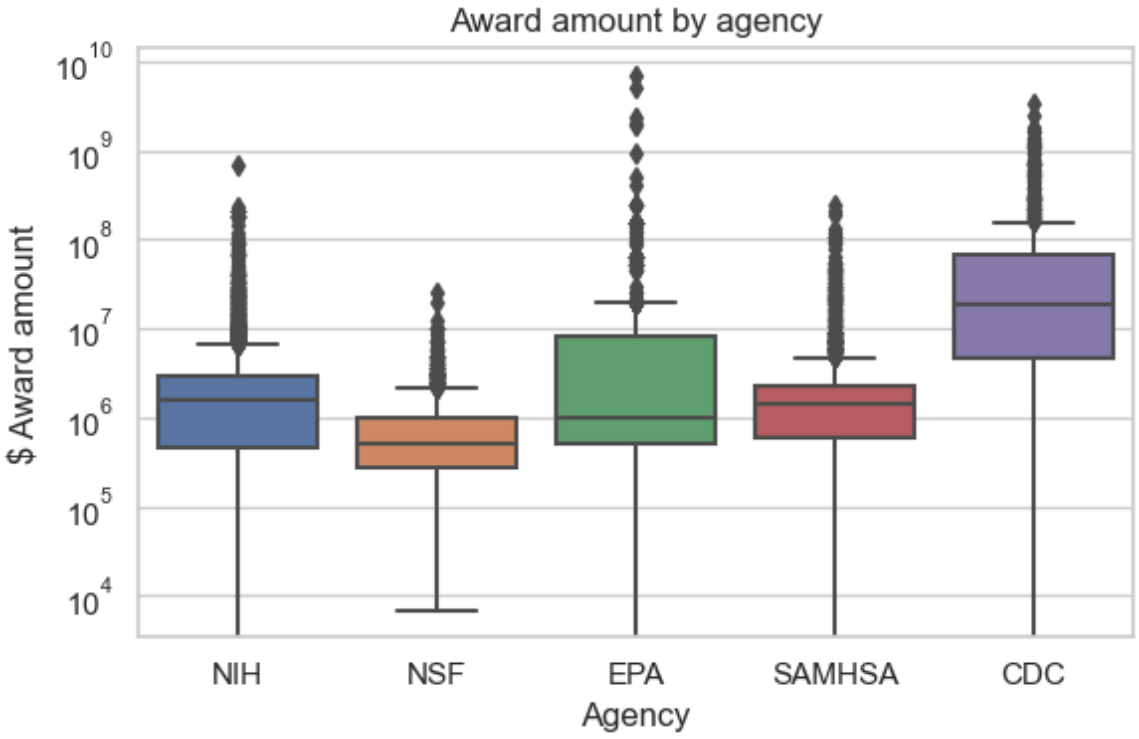
```
abstract_word_count
False    11317
True         4
Name: count, dtype: int64
title_word_count
False    11276
True        45
Name: count, dtype: int64
```

Abstract word counts (excl. Nan) have a right-skewed distribution with a peak around 400-500 words, and most abstracts are between 200-700 words. There's a tail extending up to 3000 words, but only 4 abstracts are above 1000. This might be due to a word limit for grant descriptions/abstracts.

Title word counts have more of a symmetric, bell-shaped distribution, with peak around 8 words. Only 45 are above 25 words.

In [136…

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(data=df_gw.dropna(subset=['award_amount']), x='agency',
ax.set_yscale('log')
ax.set_title('Award amount by agency')
ax.set_xlabel('Agency')
ax.set_ylabel('$ Award amount')
plt.tight_layout()
plt.show()
```



Award amounts are highest in CDC, with a wide spread. EPA has the most diverse amounts. All agencies also have some extreme outliers.

In [137…

```
df_gw[['award_amount']].describe().T.style.format('{:,.2f}')
```

Out[137…

	count	mean	std	min	25%	50%	75%	max
award_amount	12,130.00	10,027,376.95	108,581,921.73	-125,000.00	466,125.00	1,285,007.00	2,650,576.00	6,970,000,000.00

We can see that award_amount has a huge range = mean of 10m USD and standard deviation of 109m USD. A maximum value is 7 billion USD, which is probably an institutional grant. Interestingly, there's a negative minimum, meaning there could be some corrections in the data. A median of 1.3m USD is likely more representative than the mean.

Flagged words

Analysis only on df_gw_nona (without null abstracts).

In [138…
```
df_gw_nona[['num_flagged_words', 'num_flagged_words_title', 'num_fl
```

Out[138…

	count	mean	std	min	25%	50%	75%	max
num_flagged_words	11,321.00	5.36	4.26	0.00	2.00	4.00	8.00	32.00
num_flagged_words_title	11,321.00	0.49	0.84	0.00	0.00	0.00	1.00	12.00
num_flagged_words_total	11,321.00	5.85	4.72	0.00	2.00	4.00	8.00	38.00

Flagged words have an average of 5.9 per abstract, up to 32. Titles have less flagged words - mean of 0.49, which is expected given their length. About 25% grants have 0 flaged words in both abstract and title.

In [139…
```
flag_cols = [c for c in df_gw_nona.columns if c.startswith(('has_fl
summary = df_gw_nona.groupby('agency')[flag_cols].mean()
summary.insert(0, 'rows', df_gw_nona.groupby('agency').size())
for col in [c for c in flag_cols if c.startswith('has_flagged')]:
    summary[col] *= 100
summary
```

Out[139…

agency	rows	has_flagged_word	has_flagged_word_title	has_flagged_word_total	n_banned_terms	n_banned_terms_title	n_bann
EPA	671	93.889717	23.845007	94.485842	18.146051	0.406855	
NIH	5990	94.891486	21.452421	94.941569	9.602671	0.570117	
NSF	1991	92.265193	49.924661	92.415871	24.081868	1.485183	
SAMHSA	2669	97.564631	48.482578	97.677033	21.009741	1.297115	

For each agency, we see that 92-98% of abstracts have at least one flagged word, with SAMHSA being the highest and EPA lowest.

NSF has the highest number of unique flagged words in both abstract and title. EPA has the highest total count of banned terms, which the terms likely repeat. NSF and SAMHSA have a higher share of titles with flagged words - likely language used is more explicit with sensitive terminology.

Target variable analysis

In [140…
```
counts_df = (
    df_gw_nona
    .groupby('agency')['status']
    .value_counts()
    .unstack(fill_value=0)
    .T
    .sort_index()
)
```

```
counts_df
```

Out [140…

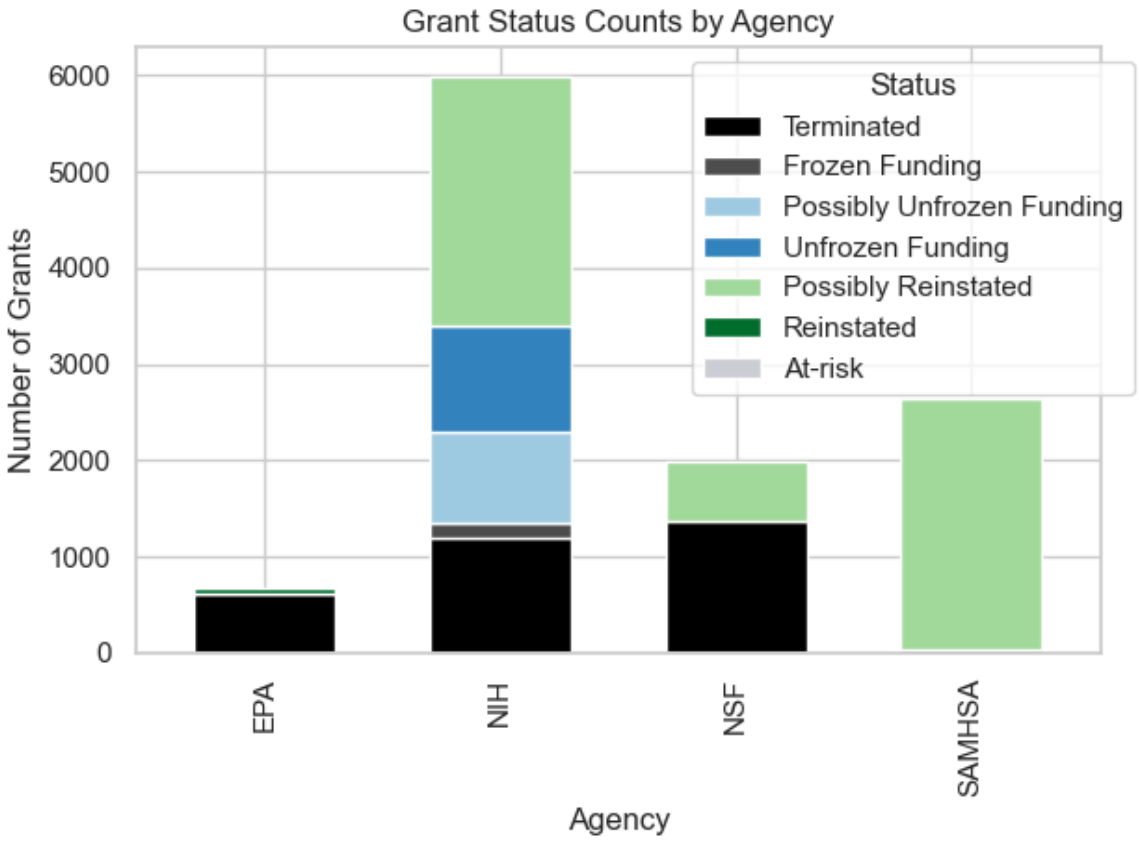
agency status	EPA	NIH	NSF	SAMHSA
Frozen Funding	0	157	0	0
Possibly Reinstated	10	2584	629	2604
Possibly Unfrozen Funding	0	951	0	0
Reinstated	54	0	0	24
Terminated	607	1192	1362	41
Unfrozen Funding	0	1106	0	0

In [141…

```
color_map = {
    "Terminated": "black",
    "Frozen Funding": "#4d4d4d",
    "Possibly Unfrozen Funding": "#9ecae1",
    "Unfrozen Funding": "#3182bd",
    "Possibly Reinstated": "#a1d99b",
    "Reinstated": "#006d2c",
    "At-risk": "#cccdd4",
}
status_order = ["Terminated", "Frozen Funding", "Possibly Unfrozen
counts_df = counts_df.reindex(status_order).fillna(0)
```

In [142…

```
fig, ax = plt.subplots()
counts_df.T.plot(
    kind='bar', stacked=True,
    color=[color_map[s] for s in counts_df.index],
    ax=ax, legend=False, width=0.6
)
ax.set_ylabel("Number of Grants")
ax.set_xlabel("Agency")
ax.set_title("Grant Status Counts by Agency")
ax.legend(counts_df.index, title="Status", bbox_to_anchor=(1.05, 1)
plt.tight_layout()
plt.show()
```



`status` column contains seven possible options: Terminated, Frozen Funding, Possibly Unfrozen Funding, Unfrozen Funding, Possibly Reinstated, and Reinstated.

As the values are not uniform across the agencies, a single target binary `target_adverse` has been created - Terminated and Frozen fundings are clear adverse outcomes with value of 1, the other outcomes are 0.

In [143…
```
adverse_statuses = ['Terminated', 'Frozen Funding']

df_gw['target_adverse'] = df_gw['status'].isin(adverse_statuses).as
df_gw_nona['target_adverse'] = df_gw_nona['status'].isin(adverse_st
```

In [144…
```
df_gw_nona['target_adverse'].value_counts()
```

Out[144…
```
target_adverse
0    7962
1    3359
Name: count, dtype: int64
```

There is clear class imbalance, with non-adverse outcomes being 70% of the dataset.

In [145…
```
df_gw.groupby('agency')['target_adverse'].value_counts().unstack(fi
```

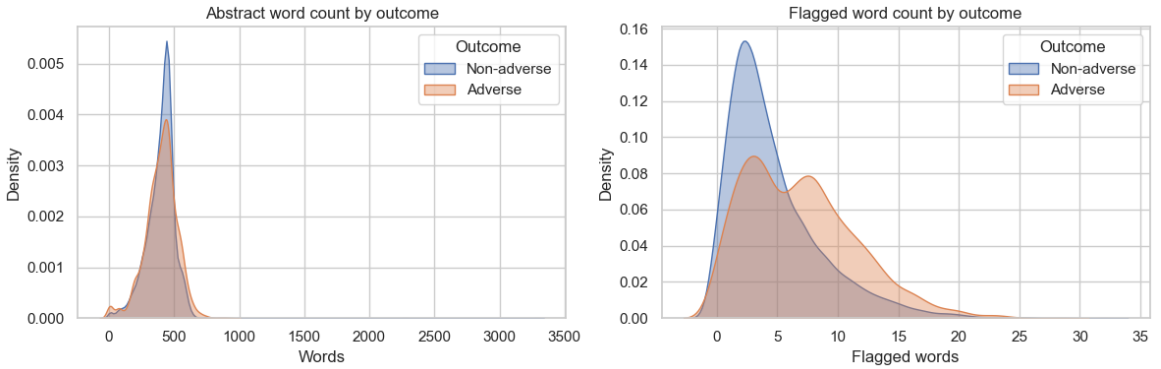
Out[145…

target_adverse agency	0	1
CDC	291	251
EPA	64	607
NIH	4652	1364
NSF	633	1363
SAMHSA	2797	134

There are differences between agencies: EPA and NSF have high adverse rates, and SAMHSA and NIH are largely unaffected. Agency is a strong single predictor can mislead the model to predict not adverse and score a 70% accuracy, which would be a failure on EPA and NSF grants.

In [146…

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for val, label in [(0, 'Non-adverse'), (1, 'Adverse')]:
    subset = df_gw_nona[df_gw_nona['target_adverse'] == val]
    sns.kdeplot(data=subset, x='abstract_word_count', fill=True, al
    sns.kdeplot(data=subset, x='num_flagged_words', fill=True, alpha

axes[0].set_title('Abstract word count by outcome')
axes[0].set_xlabel('Words')
axes[0].legend(title='Outcome')

axes[1].set_title('Flagged word count by outcome')
axes[1].set_xlabel('Flagged words')
axes[1].legend(title='Outcome')

plt.tight_layout()
plt.show()
```



Abstract length looks almost the same for both outcomes. For flagged word counts there is a clear separation, with adverse grants having a flatter distribution. Non-adverse grants cluster around 2-3, and adverse ones spread above 10.

Do grants with flagged content get terminated more often?

In [147…

```
df_gw_nona.groupby('status')[['num_flagged_words', 'num_flagged_wor
```

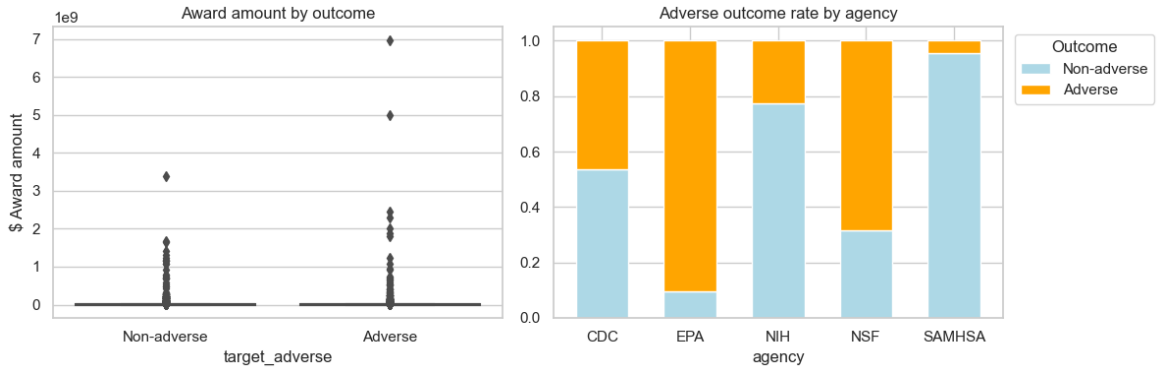
Out[147…

status	num_flagged_words	num_flagged_words_title
Frozen Funding	2.923567	0.108280
Possibly Reinstated	5.338596	0.521881
Possibly Unfrozen Funding	3.019979	0.129338
Reinstated	2.358974	0.538462
Terminated	7.088070	0.677077
Unfrozen Funding	3.073237	0.150995

Terminated grants have the highest number of banned words, and Frozen Funding and Reinstated have fewest. In titles, Terminated and (interestingly) Reinstated are the highest, with Possibly Unfrozen Funding, Unfrozen Funding

and Frozen Funding being closest to 0.

In [148…
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(data=df_gw.dropna(subset=['award_amount']), x='target_a
axes[0].set_title('Award amount by outcome')
axes[0].set_xticklabels(['Non-adverse', 'Adverse'])
axes[0].set_ylabel('$ Award amount')

ct = pd.crosstab(df_gw['agency'], df_gw['target_adverse'], normaliz
ct.columns = ['Non-adverse', 'Adverse']
ct.plot(kind='bar', stacked=True, color=['lightblue', 'orange'], ax
axes[1].set_title('Adverse outcome rate by agency')
axes[1].tick_params(axis='x', rotation=0)
axes[1].legend(title='Outcome', bbox_to_anchor=(1.01, 1), loc='uppe

plt.tight_layout()
plt.show()
```



Award amount is not a good signal for outcome - there is no clear separation.

For agency, previously described patterns are confirmed.

What do embeddings reveal about the target variable?

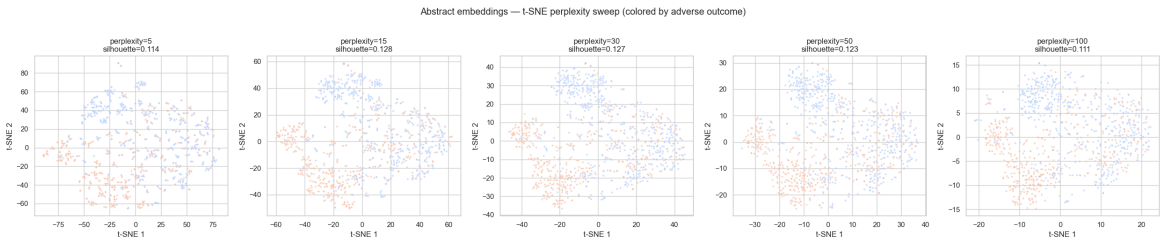
In [149…
```
def parse_embedding(x):
    if isinstance(x, str):
        return np.fromstring(x.replace('\n', ' ').strip('[]'), sep=
    return x

df_gw_nona.loc[:, 'embeddings_abstract_allminilm'] = df_gw_nona['em
df_gw_nona.loc[:, 'embeddings_title_allminilm'] = df_gw_nona['embed
df_gw['embeddings_title_allminilm'] = df_gw['embeddings_title_allmi
```

In [150…
```
n = 500
df_plot_abs = pd.concat([
    df_gw_nona[df_gw_nona['target_adverse'] == 1].sample(n, random_
    df_gw_nona[df_gw_nona['target_adverse'] == 0].sample(n, random_
])
X_abs = np.vstack(df_plot_abs['embeddings_abstract_allminilm'].toli
X_tsne_abs = TSNE(n_components=2, perplexity=30, random_state=42, i
df_plot_abs['tsne_x'] = X_tsne_abs[:, 0]
df_plot_abs['tsne_y'] = X_tsne_abs[:, 1]
```

```
df_plot_title = df_gw.dropna(subset=['embeddings_title_allminilm'])
X_title = np.vstack(df_plot_title['embeddings_title_allminilm'].tol
X_tsne_title = TSNE(n_components=2, perplexity=30, random_state=42,
df_plot_title['tsne_x'] = X_tsne_title[:, 0]
df_plot_title['tsne_y'] = X_tsne_title[:, 1]
```

In [151…

```
from sklearn.metrics import silhouette_score

perplexities = [5, 15, 30, 50, 100]
X_abs_full = np.vstack(df_plot_abs['embeddings_abstract_allminilm']
labels = df_plot_abs['target_adverse'].values

fig, axes = plt.subplots(1, len(perplexities), figsize=(5 * len(per

for ax, perp in zip(axes, perplexities):
    X_tsne = TSNE(n_components=2, perplexity=perp, random_state=42,
                  init='pca', learning_rate='auto').fit_transform(X
    sil = silhouette_score(X_tsne, labels)
    sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1],
                    hue=labels, palette='coolwarm', s=8, alpha=0.6,
                    ax=ax, legend=False)
    ax.set_title(f'perplexity={perp}\nsilhouette={sil:.3f}')
    ax.set_xlabel('t-SNE 1')
    ax.set_ylabel('t-SNE 2')

plt.suptitle('Abstract embeddings — t-SNE perplexity sweep (colored
plt.tight_layout()
plt.show()
```



For t-SNE, silhouette scores for perplexities 5-100 are around 0.11-0.13, which means there is weak but stable separation. Embeddings might be a useful feature in combination with other features, not on their own.

Data Rescue Project

In [152…

```
print(df_drp.shape)
```

```
(2539, 106)
```

DRP data with metadata from DataCite contains 2539 rows and 106 columns.

Missing data

In [153…

```
missing = df_drp.isna().mean().sort_values(ascending=False)
print(f'{(missing > 0).sum()} / {len(missing)} columns have missing
```

```
missing.head(10).to_frame('missing_rate').style.format('{:.1%}')
```

```
25 / 106 columns have missing values
```

Out [153…

	missing_rate
dc_error	100.0%
dc_rel_citations	96.7%
flagged_words_nyt_title	92.5%
flagged_words_pen_title	83.9%
flagged_words_all_title	83.6%
notes	73.2%
flagged_words_nyt	68.6%
dc_geo_places	64.4%
dc_date_collected	61.2%
flagged_words_pen	56.5%

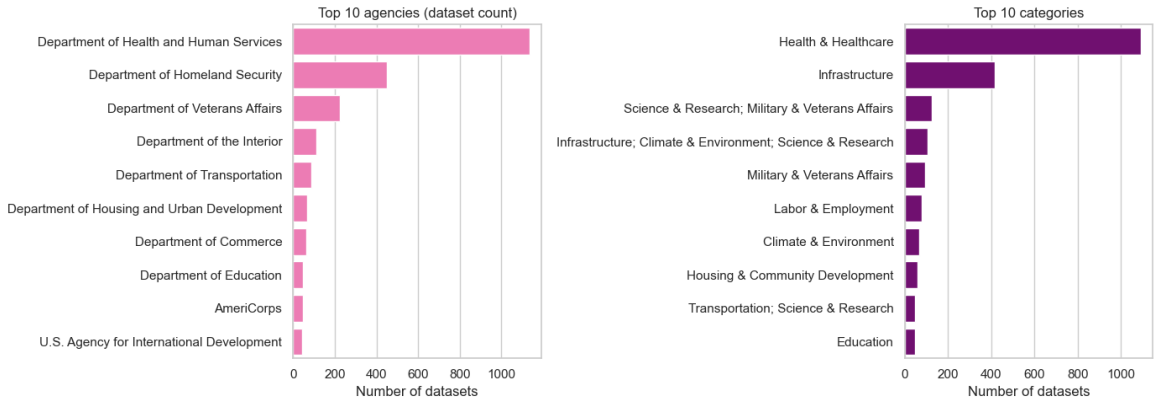
25 columns in the DRP dataset have missing values. Most missing data is in metadata fields from DataCite (for example `notes`, `geo_places`, `flagged_words_nyt`) and fields created with detection of flagged words (signaling there's no flagged words for these records).

Most important fields, like `title`, `agency`, `dc_abstract`, are fully present. `dc_abstract` has 0% missing values because DataCite was queried specifically for datasets with an abstract.

Agencies and categories

In [154…

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Top agencies
agencies = df_drp['agency'].value_counts()
top_agencies = agencies.head(10)
sns.barplot(x=top_agencies.values, y=top_agencies.index, ax=axes[0]
axes[0].set_title('Top 10 agencies (dataset count)')
axes[0].set_xlabel('Number of datasets')
axes[0].set_ylabel('')

# Top categories
cats = df_drp['category'].value_counts()
top_cats = cats.head(10)
sns.barplot(x=top_cats.values, y=top_cats.index, ax=axes[1], color=
axes[1].set_title('Top 10 categories')
axes[1].set_xlabel('Number of datasets')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()
```



In [155…
```
print(f'Agencies: {len(agencies)}, only one: {(agencies == 1).mean(
print(f'Top 1: {top_agencies.index[0], top_agencies.iloc[0]}')

print(f'Categories: {len(cats)}, only one: {(cats == 1).mean():.3g}
print(f'Top 1: {top_cats.index[0], top_cats.iloc[0]}')
```

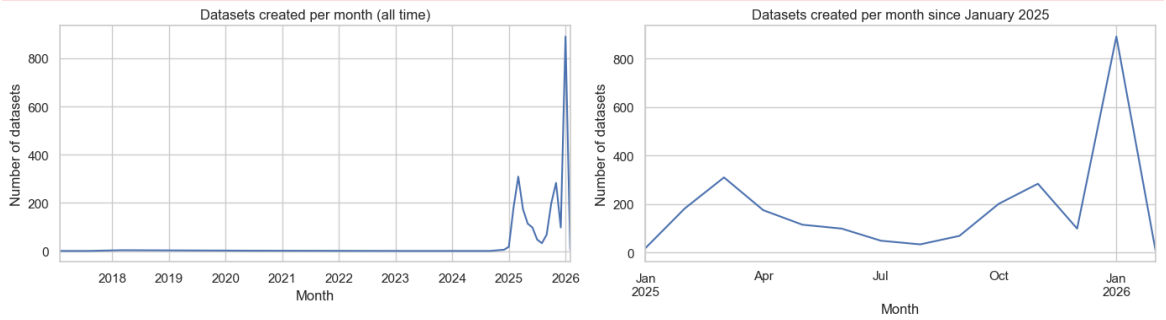
```
Agencies: 34, only one: 0.147
Top 1: ('Department of Health and Human Services', 1135)
Categories: 44, only one: 0.205
Top 1: ('Health & Healthcare', 1091)
```

DRP data includes 34 federal agencies in total, wich 14.7% being represented by only one example. Department of Health and Human Services is the largest one, with 1135 examples.

These represent a total of 44 categories of datasets, with the most common one being Health & Healthcare (1091 samples).

In [156…
```
dc_created = pd.to_datetime(df_drp['dc_created'], errors='coerce').

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

dc_created.value_counts().sort_index().plot(ax=axes[0])
axes[0].set_title('Datasets created per month (all time)')
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Number of datasets')

dc_created[dc_created >= '2025-01'].value_counts().sort_index().plo
axes[1].set_title('Datasets created per month since January 2025')
axes[1].set_xlabel('Month')
axes[1].set_ylabel('Number of datasets')

plt.tight_layout()
plt.show()
```

```
/var/folders/f0/g61h031s7yscf43_m6xj3l0c0000gn/T/ipykernel_98469/184
5899011.py:1: UserWarning: Converting to PeriodArray/Index represent
ation will drop timezone information.
  dc_created = pd.to_datetime(df_drp['dc_created'], errors='coerc
e').dt.to_period('M')
```



Most datasets in DRP have a publishing date in 2025, which indicates that it's not the original publishing date but a date of republishing on DRP - it conceals some of the original dataset information.

In [157…
```
df_drp['dc_isActive'].value_counts()
```

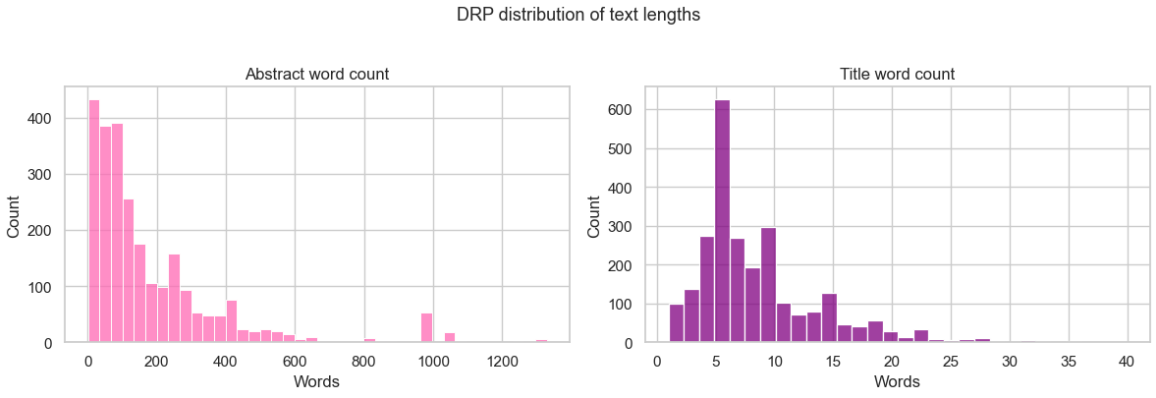
Out[157…
```
dc_isActive
True    2539
Name: count, dtype: int64
```

Activity flag is true for all datasets, also because of using the republished DOI, not the original one.

Textual features

In [158…
```
df_drp['abstract_word_count'] = df_drp['dc_abstract'].fillna('').as
df_drp['title_word_count'] = df_drp['title'].fillna('').astype(str)
```

In [159…
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(data=df_drp[df_drp['abstract_word_count'] <= 1500], x=
axes[0].set_title(f'Abstract word count')
axes[0].set_xlabel('Words')

sns.histplot(data=df_drp, x='title_word_count', bins=30, ax=axes[1]
axes[1].set_title('Title word count')
axes[1].set_xlabel('Words')

plt.suptitle('DRP distribution of text lengths', fontsize=13, y=1.0
plt.tight_layout()
plt.show()
```



Abstract word counts range from 1 to 9262 words, with mean of 195.8. The distribution is heavily right-skewed - data is dominated by shorted abstracts. There's 8 outliers (above 1500), not included on the plot for readability.

Titles word counts range from 1 word to 40, with mean of 8.5. The distribution is also right-skewed, but not as heavily as abstracts.

Flagged words

In [160…
```
counts_flagged = df_drp['has_flagged_word_total'].value_counts().re
print(counts_flagged)
print(f'\nProportion flagged: {counts_flagged.get("Flagged", 0) / l
```

```
has_flagged_word_total
Not flagged    1359
Flagged        1180
Name: count, dtype: int64

Proportion flagged: 0.46
```

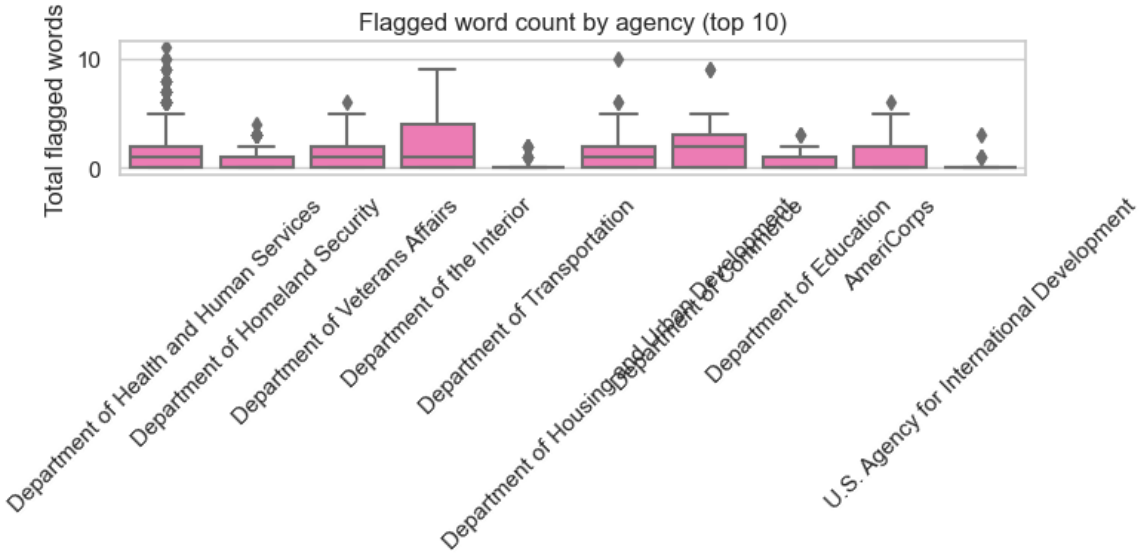
Only 46.5% of datasets in DRP contain flagged words, which makes sense considering DRP data gathering methodology. They rely not only on banned terms identified by journalists and scientists, but also on tips from insiders or specific agencies targeted by the government, so a lot of rescued data wouldn't necessarily contain sensitive language.

In [161…
```
num_cols_drp = ['abstract_word_count', 'title_word_count',
                'num_flagged_words', 'num_flagged_words_title', 'num_
                'dc_viewCount', 'dc_downloadCount', 'dc_citationCount
df_drp[[c for c in num_cols_drp if c in df_drp.columns]].describe()
```

Out[161…

	count	mean	std	min	25%	50%	75%	max
abstract_word_count	2,539.00	195.76	329.61	1.00	53.00	108.00	244.00	9,262.00
title_word_count	2,539.00	8.49	5.29	1.00	5.00	7.00	11.00	40.00
num_flagged_words	2,539.00	0.91	1.31	0.00	0.00	0.00	2.00	10.00
num_flagged_words_title	2,539.00	0.25	0.66	0.00	0.00	0.00	0.00	5.00
num_flagged_words_total	2,539.00	1.17	1.71	0.00	0.00	0.00	2.00	11.00
dc_viewCount	2,539.00	2.06	30.26	0.00	0.00	0.00	0.00	1,170.00
dc_downloadCount	2,539.00	0.41	7.80	0.00	0.00	0.00	0.00	361.00
dc_citationCount	2,539.00	0.05	0.33	0.00	0.00	0.00	0.00	10.00

In [162…
```
fig, ax = plt.subplots(figsize=(8, 4))

top_agency_names = df_drp['agency'].value_counts().head(10).index
drp_top = df_drp[df_drp['agency'].isin(top_agency_names)]

sns.boxplot(data=drp_top, x='agency', y='num_flagged_words_total', 
ax.tick_params(axis='x', rotation=45)
ax.set_title('Flagged word count by agency (top 10)')
ax.set_xlabel('')
ax.set_ylabel('Total flagged words')
plt.tight_layout()
plt.show()
```

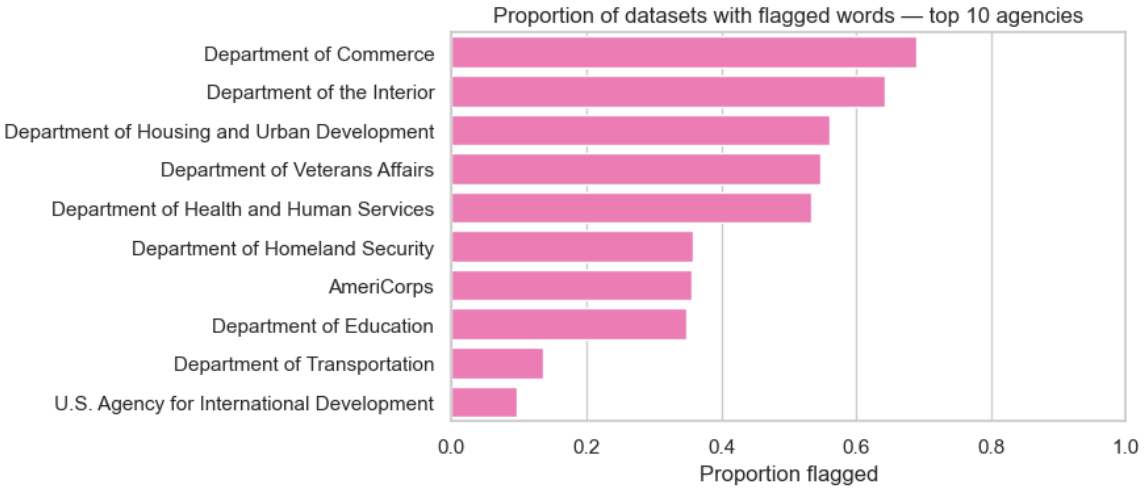


Numbers of flagged words are similar across agencies - Departament of

Interior has a higher median, with Departament of Transportation and US Agency for International Development having the lowest. Department of Health and Services has the most outliers.

In [163…

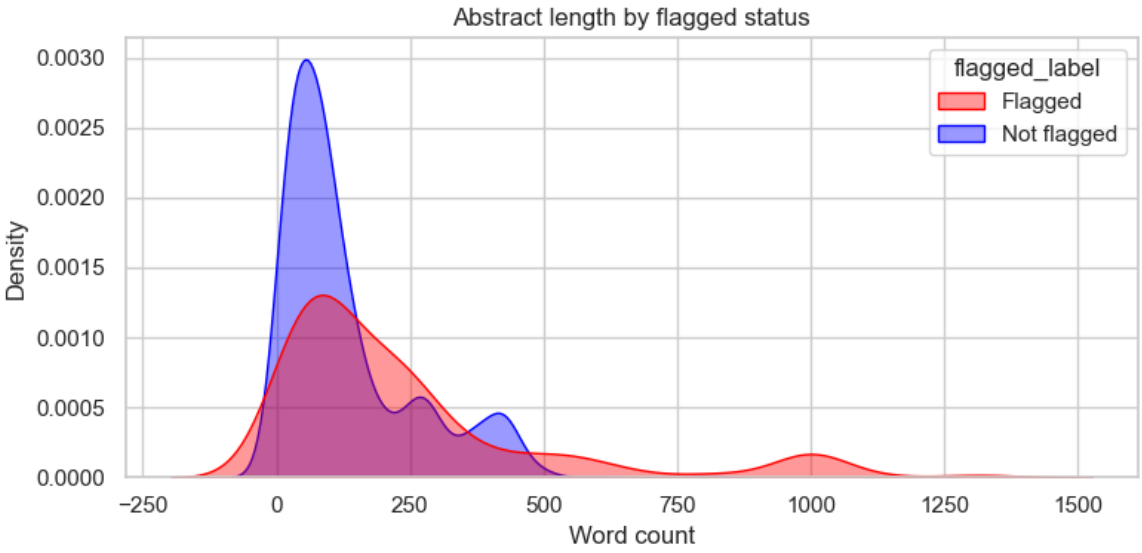
```
flag_by_agency = (
    drp_top.groupby('agency')['has_flagged_word_total']
    .mean()
    .reindex(top_agency_names)
    .sort_values(ascending=False)
)

fig, ax = plt.subplots(figsize=(9, 4))
sns.barplot(x=flag_by_agency.values, y=flag_by_agency.index, ax=ax,
ax.set_title('Proportion of datasets with flagged words — top 10 ag
ax.set_xlabel('Proportion flagged')
ax.set_ylabel('')
ax.set_xlim(0, 1)
plt.tight_layout()
plt.show()
```



The only two departments for which over 50% of data has flagged words is the Department of Commerce and Department of Interior.

In [164…

```
fig, ax = plt.subplots(figsize=(8, 4))

df_drp['flagged_label'] = df_drp['has_flagged_word_total'].map({0:
drp_capped = df_drp[df_drp['abstract_word_count'] <= 1500]

sns.kdeplot(data=drp_capped, x='abstract_word_count', hue='flagged_
            fill=True, alpha=0.4, ax=ax,
            palette={'Not flagged': 'blue', 'Flagged': 'red'})

ax.set_title('Abstract length by flagged status')
ax.set_xlabel('Word count')
ax.set_ylabel('Density')
plt.tight_layout()
plt.show()
```



There is an overlap between flagged and non-flagged abstracts, but the shapes are noticably different. Non-flagged abstracts are shorter with peak around 75 words. Flagged abstracts spread out more evenly up to 500+ words. However, longer abstracts naturally also provide more opportunity for flagged words to appear, which might be confusing for the future model.

The metadata based on DOI identifiers from DRP is related to republishing, and some of it's abstracts/notes reveal that it has been included in the project and introduces leakage to any modeling that would utilize textual features.

To address this, each DRP dataset is being matched to its original state using Data.gov, the largest federal repository of datasets, and its Harvard mirror. This will provide pre-rescue metadata as model features and a better basis for constructing negative examples.